Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- n^2 karmaşıklıklı sıralama algoritmalarının karşılaştırması (tekrar)
- Insertion Sort'un Selection Sort'a göre neden daha avantajlı olduğu
- Shell Sort: adım büyüklüğü $h = N/3$ ile ayrıntılı örnek
- Shell Sort'un akış diyagramı ve `tmp` ile iyileştirilmiş hâli
- Hangi sıralama algoritmasının ne zaman seçileceği
- Algoritmaların çalışma süresini deneysel olarak ölçme önerisi

Hocanın Özellikle Vurguladığı Kısımlar

- Bubble Sort'un dezavantajı, her karşılaştırmanın sonunda yer değiştirme yapmak zorunda kalmasıdır. Yer değiştirme (exchange) masraflı bir işlemdir.
- Bu dönem yalnız "çalışan" değil, verimli algoritmalar yazmaya dikkat ediliyor. Yazılan algoritmanın nasıl iyileştirileceği düşünülmelidir.
- Insertion Sort'un en avantajlı olduğu durum yarı sıralı dizilerdir.
- 12 elemanlı bir dizide hangi algoritmanın kullanıldığı önemli değildir. Fark, n çok büyüdüğünde ortaya çıkar.
- Tek bir "doğru" algoritma yoktur. Her algoritmanın avantajları ve dezavantajları bilinmeli, seçim buna göre yapılmalıdır.
- Algoritmaların zamanı ve yeri iyi kullanması gittikçe daha önemli hâle geliyor. Bilgisayar mühendisi verimli çözümler sunmalıdır.

Kısa Tekrar Notları

- Bubble, Selection ve Insertion Sort'un en kötü ve ortalama durum karmaşıklığı n^2'dir.
- En iyi durumda Bubble Sort (erken çıkışlı) ve Insertion Sort n karmaşıklığa iner. Selection Sort'ta en iyi durum da n^2'dir.
- **Insertion Sort vs Selection Sort:** Selection Sort her turda kalan elemanların hepsine bakıp en küçüğü bulmak zorundadır. Insertion Sort ise sırası gelen elemanın yalnız baş tarafa göre yerini arar ve yerini bulunca durur.
- **Shell Sort:**
  - Adım büyüklüğü h ile birbirinden h uzaklıktaki elemanlar kendi içinde Insertion Sort ile sıralanır.
  - Her turda h küçültülür. Son tur ($h = 1$) klasik Insertion Sort'tur.
  - İyi bir adım dizisiyle en kötü durum $n^{3/2}$ civarındadır. Bazı adım dizileriyle $n^{4/3}$ veya $n^{5/4}$ gibi değerler elde edilir.
- Shell Sort, orta büyüklükteki (yüzlerce elemanlı) ve yarı sıralı dizilerde avantajlıdır. Milyonlarca eleman için Quick Sort gibi $n \log n$ algoritmalar kullanılmalıdır.

Detaylı Açıklamalar

n^2 Algoritmaların Karşılaştırması

Önceki derste Bubble Sort, Selection Sort, Insertion Sort ve Shell Sort anlatılmıştı. Hepsi en kötü durumda n^2 karmaşıklıkla çalışır. O hâlde hangisinin kullanıldığı fark eder mi?

- En iyi durumlarla pek ilgilenilmez. Çünkü en iyi durum çok nadir karşılaşılan bir durumdur.
- Genelde ortalama durumla karşılaşılır. Dizi ne küçükten büyüğe ne büyükten küçüğe sıralıdır, karışıktır.
- Ortalama durumda da Bubble, Selection ve Insertion Sort'un karmaşıklığı n^2'dir.

Bubble Sort neden kötü bilinir?

- Ardışık elemanlara ikişer ikişer bakar. Gerektikçe sürekli yer değiştirmek zorundadır.
- Diğer algoritmalar da sıralamak için yer değiştirir. Ama önce bir takım kontroller yapar, sonra "bunun yerini değiştirelim" der.
- Yer değiştirme bir değiş tokuş (exchange) işlemidir: A'daki B'nin gözüne, B'deki A'nın gözüne yazılır. Bu yüzden masraflıdır.

Insertion Sort Neden Selection Sort'tan Daha Avantajlı?

Insertion Sort'ta baş taraf kendi içinde sıralıdır. Örneğin `3 5 7` yerleşmiş, sırada 6 olsun:

- 6, 7'den küçük. 7 bir sağa kayar.
- 6, 5'ten küçük değil. O hâlde 3'ten de küçük değildir. 3'e bakmaya bile gerek yoktur.

Hoca şu soruyu sordu: Selection Sort da yerleştirdiği kısma bir daha bakmaz. O hâlde Insertion Sort neden hâlâ daha avantajlı?

- Bir öğrencinin cevabı: Selection Sort'ta kalan elemanların hepsi arasında en küçüğü bulmak gerekir. Insertion Sort'ta ise hangi elemandan başlanacağı bellidir. Bir adım sonra bile durulabilir.
- Hoca bu cevabı onayladı. Insertion Sort, tüm elemanlar arasında en küçüğü arama derdinde değildir. Yalnız sırası gelen elemanın baş tarafa göre yerini belirler ve bulduğu noktada durur.
- Fark çok büyük değildir. Ama Insertion Sort'un n^2 algoritmalar içinde tercih edilmesinin sebebi budur.

Insertion Sort'un en avantajlı olduğu durum **yarı sıralı dizilerdir**: Dizi zamanında sıralıymış, bazı işlemlerden sonra sırası bozulmuş. Bu durumda sıralarken çok fazla eleman hareketi olmaz.

Shell Sort: Neyi İyileştiriyor?

Insertion Sort'ta küçük bir eleman sonlarda kaldıysa (örneğin dizi tersten sıralıysa), onu başa taşımak için tek tek tüm elemanları geçmek gerekir. Bu bir dezavantaj değil, iyileştirilebilecek bir noktadır.

Shell Sort'un fikri: Elemanları birer birer değil, **zıplaya zıplaya** karşılaştırmak.

- Sondaki bir eleman, örneğin 5 öncesindeki elemanla karşılaştırılır.
- Ondan küçükse, onun 5 öncesindekiyle karşılaştırılır.
- Böylece küçük eleman başa çok daha hızlı taşınır.

Adım büyüklüğü (step size) seçimi:

- Adım büyüklüğünün nasıl seçileceğine dair farklı yöntemler ve çalışmalar vardır.
- $N/2$ alanlar da vardır, farklı adım büyüklükleri kullananlar da.
- Bu derste yaygın yöntemlerden biri olan $h = N/3$ kullanıldı. Her turda h 3'e bölünür.

Shell Sort Örneği ($N = 12$, $h = 4$)

Dizi (ekrandan, 00:25:00):

```
indeks:  0   1   2   3   4   5   6   7   8   9  10  11
dizi  : 21   8  13  19  16  11   7  25  12  10   5   4

N = 12, h = N/3 = 4
Kendi içinde sıralanacak alt diziler (4 atlayarak):
  0, 4,  8 -> 21 16 12  -> 12 16 21
```

```
  1, 5,  9 ->  8 11 10  ->  8 10 11
  2, 6, 10 -> 13  7  5  ->  5  7 13
  3, 7, 11 -> 19 25  4  ->  4 19 25

h = 4 sonunda: 12  8  5  4  16  10  7  19  21  11  13  25
```

Adım adım ilerleyiş:

- $i = 4$: 16, 4 gerisindeki 21 ile karşılaştırılır. 16 küçük, yer değiştirir. 0'ın 4 gerisi olmadığı için durulur.
- $i = 5$: 11, 8 ile karşılaştırılır. 11 büyük, değişmez.
- $i = 6$: 7, 13 ile karşılaştırılır. 7 küçük, yer değiştirir.
- $i = 7$: 25, 19 ile karşılaştırılır. Değişmez.
- $i = 8$: 12, önce 21 ile, sonra 16 ile karşılaştırılır. İkisinden de küçük. 12 en başa gelir.
- $i = 9$: 10, 11 ile karşılaştırılır, yer değiştirir. Sonra 8 ile karşılaştırılır, durur.
- $i = 10$: 5, önce 13 ile, sonra 7 ile karşılaştırılır. 2. göze gelir.
- $i = 11$: 4, önce 25 ile, sonra 19 ile karşılaştırılır. 3. göze gelir.

Hoca karşılaştırılacak elemanları aynı renge boyadı. Her eleman yalnız kendi rengindeki elemanlarla karşılaştırıldı.

Kazanç:

- 10. gözdeki 5, yalnız 2 karşılaştırmayla (13 ve 7) 2. göze geldi.
- Son gözdeki 4, yalnız 2 karşılaştırmayla (25 ve 19) 3. göze geldi.
- Insertion Sort'ta 4'ü başa taşımak için 11. gözden başa kadar tüm elemanlarla tek tek karşılaştırmak gerekirdi.

Sonraki tur: $h = 4/3 = 1$. Bu tur artık klasik Insertion Sort'tur. 8, 12'den küçük; 5 ve 4 başa doğru kaydırılır vb.

Büyük dizide adımlar: $N = 1200$ ve adım büyüklüğü 3 ise:

- İlk turda $h = 400$. Elemanlar 400'er ileriye bakılarak karşılaştırılır.
- Sonra h yaklaşık 130 olur. Sonra 40 civarı, ... en son 1.
- Sonlarda kalan küçük bir eleman tek seferde 400 adım öne gelebilir.

İki Farklı Gezinme Sırası

Bir öğrenci önceki derste farklı bir sıra izlendiğini sordu. Önceki derste aynı alt dizinin elemanlarına arka arkaya bakılmıştı (4, 8, 12, ...). Bu derste ise i birer birer artırıldı (4, 5, 6, ...).

- Hoca ikisinin de doğru olduğunu söyledi. İkisi de birebir aynı işi yapar, karmaşıklıkları da aynıdır.
- Bu derste i'yi birer artırmasının sebebi akış diyagramının daha rahat anlaşılmasıdır.

Shell Sort Akış Diyagramı

Akış diyagramı (ekrandan, 00:51:30):

```
Giriş: N, a[N], stepSize
h = N / stepSize
h >= 1 olduğu sürece:
    i = h'den N-1'e kadar (birer artarak):
        j = i
        (j >= h) && (a[j] < a[j-h]) olduğu sürece:
            a[j] <-> a[j-h]
            j = j - h
    h = h / stepSize
Çıkış: a[N]
```

- İç döngü şartında $j \geq h$ gerekir. Çünkü bir elemanı kendisinden h önceki elemanla karşılaştırabilmek için o gözün var olması gerekir. $h = 4$ ise $j = 3$ için 4 öncesi yoktur.

- Örnek: $j = 5$ ise 5. göz 1. gözle karşılaştırılır. Yer değiştirme olursa j 4 azalır ve 1 olur. 1'in 4 gerisi olmadığı için döngüden çıkılır.
- Örnek: $j = 10$ ise önce 6. gözle, sonra 2. gözle karşılaştırılır. Sonra döngü biter.
- Dizi büyükten küçüğe sıralıysa bütün bu karşılaştırmalar yapılmak zorundadır. Hoca bunu `21 15 14 13 8 7 7 6 5 4 3 2` dizisi üzerinde gösterdi.

Bir tur bitince ne yapılmalı? Bir öğrencinin cevabı: h, adım büyüklüğüne bölünmeli. Bu işlem $h = 1$ olana kadar sürer.

İyileştirme: Yer değiştirme (exchange) yerine Insertion Sort'taki gibi kaydırma yapılabilir:

- Döngüden önce `tmp = a[j]` alınır.
- Şart `tmp < a[j-h]` olur.
- Döngü içinde yalnız `a[j] = a[j-h]` yazılır.
- Döngüden çıkınca `a[j] = tmp` yazılır.

Hoca, ilk anlatımda rahat anlaşılsın diye exchange kullandığını söyledi.

C karşılığı (derste C kodu yazılmadı; akış diyagramının `tmp`'li hâli ajan tarafından C'ye çevrildi, derlenip yukarıdaki 12 elemanlı diziyle test edildi; ilk tur sonucu tahtadaki diziyle aynı çıktı):

```c
void shell_sort(int a[], int n, int step_size) {
    int h = n / step_size, i, j, tmp;
    while (h >= 1) {
        for (i = h; i <= n - 1; i++) {
            tmp = a[i];
            j = i;
            while (j >= h && tmp < a[j - h]) {
                a[j] = a[j - h];
                j = j - h;
            }
            a[j] = tmp;
        }
        h = h / step_size;
    }
}
```

Öğrenci Soruları

1. Bubble Sort'taki gibi bir `swap` bayrağı tutup, bir turda hiç yer değiştirme olmazsa durulamaz mı?

- Hoca soruyu mantıklı buldu ama emin olmadığını söyledi. Denemeden kesin konuşmak istemedi.
- Çekincesi şuydu: Her turda dizi, kendi içinde ayrı alt dizilerden oluşur. Alt dizilerin kendi içinde sıralı olması, dizinin tamamının sıralı olduğu anlamına gelmeyebilir.
- Konunun bir sonraki derste konuşulmasına karar verildi.

2. Insertion ve Selection Sort'ta her adımda baş taraf sıralı olur. Bu bir avantaj mı?

- Hoca, yapılacak işe bağlı olduğunu söyledi. Sıralı kısım bir işlem için kullanılıyorsa bu bir avantajdır.
- Shell Sort'ta baş taraf adım adım sıralı hâle gelmez.
- Hoca ayrıca şunu hatırlattı: Quick Sort normalde $n \log n$ karmaşıklıkla en iyi sıralama algoritmasıdır. Ama sıralı bir dizi verilirse n^2 karmaşıklığa düşer. Dizinin sıralı gelmesi zaten çok beklenen bir durum değildir.

3. Hangi sıralama algoritmasının daha avantajlı olduğunu bulan bir algoritma var mı?

- Algoritmalar çalıştırılıp süreleri ölçülerek karşılaştırılır.
- Durumlar çok değişir: dizi biraz sıralı mı, küçükten büyüğe mi, büyükten küçüğe mi? Bazı farklar gözle anlaşılır, bazıları boyut arttıkça ortaya çıkar.
- Örneğin en beğenilmeyen Bubble Sort, dizi sıralıysa Selection Sort'un önüne geçer.

- Merge Sort, $n \log n$ algoritmalar içinde en çok kullanılanlardan biridir. Ama yedek dizi kullanır. Çok eleman varsa ve yedek dizi için yer yoksa kullanılmamalıdır.
- Java Quick Sort kullanır. Quick Sort özyinelemelidir (recursive), diziyi küçük parçalara bölerek sıralar. Küçük parçalara inildiğinde, ya da baştan küçük bir dizi verilirse, Insertion Sort kullanır. Çünkü özyinelemeli fonksiyon çağırmanın ek yükü (overhead) vardır.

4. Akış diyagramında kısaltma kullanılabilir mi?

- Bir öğrenci, diziyi tek bir giriş kutusunda almak gibi kısaltmaların sınavlarda sorun olup olmayacağını sordu.
- Hoca, birinci sınıfta dizilerin döngüyle okunup yazıldığını açık açık gösterdiğini anlattı. Çünkü birinci sınıf öğrencisi örneğin dizinin 0'dan başladığını ve $a[n]$ diye bir eleman olmadığını oturtmakta zorlanır.
- Hocaya göre ikinci sınıf ikinci dönemde bu kısaltmaların bir sakıncası yoktur. Anlamı açık olduğu sürece kullanılabilir.

Shell Sort Ne Zaman Kullanılmalı?

- **Daha büyük dizilerde:** 12 elemanlı bir diziyi parçalara bölüp dörder dörder atlatmak gereksizdir. $n = 10$ iken fark yoktur, $n = 10^8$ iken büyük fark vardır.
- Shell Sort, karmaşıklığı $n^{3/2}$ olsa da sonuçta n^2'ye yakın bir algoritmadır. 100, 200, 300 elemanlı gibi orta büyüklükteki dizilerde uygundur.
- 1 milyon eleman varsa Quick Sort gibi algoritmalar kullanılmalıdır.
- Insertion Sort gibi, tamamen rastgele değil de biraz sıralı dizilerde avantajlıdır. Örneğin önceden sıralı olup gelen giden elemanlarla sırası hafifçe bozulmuş diziler.
- Genel olarak Insertion Sort'tan da Selection Sort'tan da hızlıdır.

Deneysel Karşılaştırma Önerisi

Hoca, algoritmaların davranışını görmek için şu denemeleri önerdi:

- Ekrana hiç yazdırma yapmadan, işlemlerin başladığı ve bittiği yere sistem zamanını ölçen bir fonksiyon koyun. Böylece çalışma süresini ölçün.
- Büyük diziler kullanın: 1000 veya 5000 eleman. Elemanları bir dosyadan okutun ya da programa rastgele sayı ürettirin.
- Aynı dizide Selection ve Insertion Sort'u çalıştırıp süreleri karşılaştırın.
- Diziyi yarıya kadar sıraladıktan sonra tekrar deneyin. Büyükten küçüğe sıralı dizi için de deneyin.
- Hocaya göre bu tür denemelere alışan kişi, ileride gerçek problemlerde farkına bile varmadan daha verimli kod yazar.

Ders sonrası notlar

- Hoca $h = 4$ için "3 tane parçam var" dedi. Aslında 4 alt dizi vardır (0-4-8, 1-5-9, 2-6-10, 3-7-11). Her alt dizide 3 eleman vardır.
- $N = 1200$ örneğinde $400/3$ tam sayı bölmesiyle 133'tür. Hoca yaklaşık olarak "130" dedi. Sonraki adım $133/3 = 44$'tür.
- **`swap` bayrağıyla erken durma sorusunun cevabı (ajan notu):** Bir h turunda hiç yer değiştirme olmaması, dizinin yalnız "h-sıralı" olduğunu gösterir. Dizi tamamen sıralı olmayabilir. Örneğin `2 1 4 3` dizisinde $h = 2$ için alt diziler (`2 4` ve `1 3`) zaten sıralıdır ve hiç yer değiştirme olmaz. Ama dizi sıralı değildir. Bu yüzden $h > 1$ turlarında erken durmak yanlış sonuç verebilir. Bu örnek ajan tarafından yukarıdaki kodla denendi. Hocanın çekincesi doğrudur.
- Quick Sort'un sıralı dizide n^2'ye düşmesi, pivotun ilk veya son eleman seçildiği basit sürüm için geçerlidir. Rastgele pivot veya üçün ortancası (median-of-three) gibi seçimlerle bu durum büyük ölçüde önlenir.
- C'de çalışma süresi ölçmek için örneğin `time.h` içindeki `clock()` fonksiyonu kullanılabilir. Altyazıda hocanın adını verdiği fonksiyon anlaşılmıyor.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.